“Design and Implementation of an Intelligent Recipe Finder and Optimization System Using Data Structures and Algorithms”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

POLISETTY SREEHARSHA :2520030123

ARNAV CHITIKENA :2520030331

HARSHITH MADDINENI :2520030573

Team Number: 13

Section: 2

Under the Guidance of

Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Design and Implementation of an Intelligent Recipe Finder and Optimization System Using Data Structures and Algorithms"** is a record of Bonafide work of **POLISETTY SREEHARSHA – 2520030123, ARNAV CHITIKENA – 2520030331, HARSHITH MADDINENI – 2520030573** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

POLISETTY SREEHARSHA – 2520030123
ARNAV CHITIKENA – 2520030331
HARSHITH MADDINENI – 2520030573

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled **"Design and Implementation of an Intelligent Recipe Finder and Optimization System Using Data Structures and Algorithms"** is a bonafide work done and submitted by **POLISETTY SREEHARSHA – 2520030123, ARNAV CHITIKENA – 2520030331, HARSHITH MADDINENI – 2520030573** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Dr. S. Vinay Kumar**, Associate Professor, Department of Computer Science Engineering, for his invaluable guidance, continuous support, and constructive feedback throughout the design and implementation of this project.

We would also like to extend our special thanks to **Dr. P. Venkateshwara Rao**, Associate Professor and Head of Department (HOD CSE), for providing us with the necessary academic infrastructure and resources to successfully execute this work. Finally, we thank KL University Hyderabad for offering us the opportunity to work on this Data Structures and Algorithms - 3 project

POLISETTY SREEHARSHA – 2520030123

ARNAV CHITIKENA – 2520030331

HARSHITH MADDINENI – 2520030573

Abstract

In the modern culinary domain, users often struggle with efficient recipe retrieval based on available ingredients, calorie restrictions, and specific cooking preferences. Existing recipe applications frequently rely on basic database queries that fail to provide optimized meal selection, string-matching resilience, or resource-allocation constraints.

To address this challenge, this project presents the design and development of an **Intelligent Recipe Finder System** implemented in Java. The application incorporates core Data Structures and Algorithms - 3 (DSA-3) concepts, including String Matching Algorithms (KMP and Rabin-Karp), Greedy Algorithms, Dynamic Programming (0/1 Knapsack Problem), Network Flow Analysis (Edmonds-Karp / Ford-Fulkerson Algorithm), and Custom Graph Representations.

The system processes rich datasets of recipes across multiple categories (including regional and global cuisines). It provides functional capabilities for exact and pattern-based ingredient matching, nutritional value optimization under target calorie limits, and network flow modeling for ingredient-recipe-user mappings. By integrating algorithmic efficiency with a modular software structure, this project delivers a fast, flexible, and resource-conscious digital solution for meal planning and recipe management.

Table of Contents

			Page No.
	Acknowledgement		4
	Abstract		6
	Table of Contents		
	List of Figures		
1	Introduction	1.1 Overview 1.2 Problem statement 1.3 Scope of the project 1.4 Objectives	9-10
2	System Requirements	2.1 Functional Requirements 2.2 Non-Functional Requirements 2.3 Hardware & Software Specifications	10
3	System Design	3.1 High-level Architecture Diagram 3.2 Description of each layer	11
4	Implementation	4.1 Module-wise implementation 4.2 Module 1: KMP Substring Search Pseudocode 4.3 Module 2: 0/1 Knapsack Optimization Pseudocode 4.4 Module 3: Edmonds-Karp Max-Flow Algorithm Pseudocode	11-14
5	Results		15-17
6	Challenges & Limitations	6.1 Issues faced during development 6.2 Solutions applied 6.3 Limitations of the System	17
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills learned during development	18
	References	- Books, tutorials, APIs, documentation sites used	18

List of Figures

3.1 High-Level Architecture Diagram of Recipe Finder System 11

5.1 System Initializing and Recipe Dataset Loading (350 Recipes Loaded) 15

5.2 Search Execution and Results Sorted using Randomized QuickSort 16

5.3 Z-Function Search Performance and Benchmark Execution Output 16

5.4 Recipe Retrieval with Detailed Preparation Steps Output 17

1. INTRODUCTION

1.1 Overview

The **Recipe Finder System** is a software application designed to help users search, filter, and optimize recipe selection based on ingredient availability and constraints. Managing large recipe datasets requires efficient data structures for storage, rapid searching, and optimization techniques for meal selection

This project implements advanced DSA concepts:

1. **String Algorithms**: KMP (Knuth-Morris-Pratt) and Rabin-Karp algorithms for substring searching across recipe names, instructions, and ingredients list.
2. **0/1 Knapsack Optimization (Dynamic Programming)**: Maximize culinary satisfaction/rating within a specified maximum caloric intake budget.
3. **Max-Flow / Network Flow Algorithms**: Edmonds-Karp / Ford-Fulkerson algorithm using BFS to compute maximum ingredient utilization and recipe mapping paths.
4. **Greedy Algorithms**: Sorting and ranking recipes efficiently based on cost, rating, or preparation time.

1.2 Problem Statement

Traditional culinary searching systems suffer from slow linear search times, lack of constraint optimization (e.g., maximizing meal utility under strict calorie bounds), and inability to model complex flow dependencies between ingredients and recipes. There is a need for an integrated system that uses optimal data structures to search, filter, and optimize recipe selection efficiently.

1.3 Scope of the Project

1) Efficient searching of recipes by name, ingredient, and cooking method.
2) Advanced string matching using KMP and Rabin-Karp algorithms.
3) Meal package optimization using the Dynamic Programming 0/1 Knapsack formulation.
4) Network flow modeling for ingredient distribution and capacity evaluation using Edmonds-Karp Max-Flow.
5) Parsing external text datasets into memory efficiently.

1.4 Objectives

1) Design and implement custom Java data models (Recipe, RecipeReader, Knapsack, MaxFlow, StringAlgorithms).
2) Provide linear and pattern-matching search functionality over large datasets.
3) Implement Dynamic Programming to select the best combination of recipes within calorie constraints.
4) Apply Network Flow algorithms to evaluate multi-source supply and recipe preparation capacities.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

1) Dataset Ingestion: The system must load and parse recipe data files (recipes.txt, recipes_01_to_50.txt, etc.) containing IDs, titles, ingredients, instructions, and nutritional information.
2) Pattern Matching & Search: Users can search for recipes using KMP and Rabin-Karp algorithms.
3) Nutritional Optimization: Given a calorie limit $\boldsymbol{W}$, the system calculates the maximum satisfaction score using 0/1 Knapsack DP.
4) Network Flow Analysis: Calculates maximum ingredient-to-recipe flow using the Edmonds-Karp algorithm.
5) Console UI: Interactive command-line menu for navigation and execution.

2.2 Non-Functional Requirements

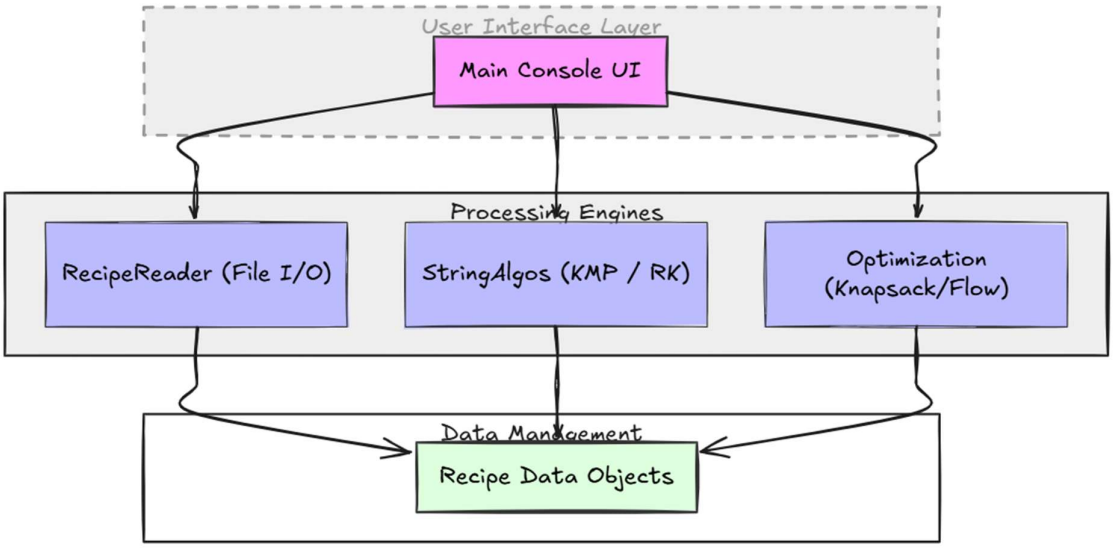
1) Performance: Substring matching completed in linear time $\boldsymbol{O(N+M)}$ using KMP.
2) Scalability: Capable of handling thousands of recipe entries in memory.
3) Usability: Clear command-line prompts and formatted output tables.
4) Reliability: Robust error handling for file I/O operations.

2.3 Hardware & Software Specifications

1) Operating System: Windows 10/11, Linux, or macOS
2) Language/Platform: Java SE Development Kit (JDK 17 or later)
3) IDE: Visual Studio Code / IntelliJ IDEA / Eclipse
4) Version Control: Git / GitHub

3. SYSTEM DESIGN

3.1 High-Level Architecture Diagram



3.2 Layer Description

 Presentation Layer (Main.java): Handles user input and displays results.

 Data Ingestion Layer (RecipeReader.java): Reads text files and creates Recipe instances.

 Core Processing & Algorithmic Layer:

- StringAlgorithms.java: Contains KMP and Rabin-Karp search logic.
- Knapsack.java: DP algorithm for calorie optimization.
- MaxFlow.java: Graph residual network and BFS flow calculations.

 Domain Layer (Recipe.java): Encapsulates attributes (ID, Title, Ingredients, Calories, Rating).

4. IMPLEMENTATION & PSEUDOCODE

4.1 Module-Wise Implementation

Module 1: Knuth-Morris-Pratt (KMP) Substring Search

Avoids redundant comparisons by computing a Longest Prefix Suffix (LPS) array.

```
ALGORITHM ComputeLPSArray(pattern, M, lps)
   len = 0, i = 1
   lps[0] = 0
   WHILE i < M DO
      IF pattern[i] == pattern[len] THEN
         len = len + 1
         lps[i] = len
         i = i + 1
      ELSE
         IF len != 0 THEN
            len = lps[len - 1]
         ELSE
            lps[i] = 0
            i = i + 1


ALGORITHM KMPSearch(pattern, text)
   M = length(pattern), N = length(text)
   ComputeLPSArray(pattern, M, lps)
   i = 0, j = 0
   WHILE i < N DO
      IF pattern[j] == text[i] THEN
         i++, j++
      IF j == M THEN
         PRINT "Pattern found at index " + (i - j)
         j = lps[j - 1]
      ELSE IF i < N AND pattern[j] != text[i] THEN
         IF j != 0 THEN
```

```
        j = lps[j - 1]
      ELSE
        i = i + 1
```

Module 2: 0/1 Knapsack Optimization (Dynamic Programming)

Selects recipes to maximize satisfaction under a maximum calorie constraint.

```
ALGORITHM SolveKnapsack(weights[], values[], N, W)
  DECLARE dp[N + 1][W + 1]

  FOR i FROM 0 TO N DO
    FOR w FROM 0 TO W DO
      IF i == 0 OR w == 0 THEN
        dp[i][w] = 0
      ELSE IF weights[i - 1] <= w THEN
        dp[i][w] = MAX(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w])
      ELSE
        dp[i][w] = dp[i - 1][w]

  RETURN dp[N][W]
```

Module 3: Edmonds-Karp Max-Flow Algorithm

Uses BFS to find augmenting paths in the residual graph.

```
ALGORITHM BFS(residualGraph, source, sink, parent)
  VISITED = array of size V initialized to false
  QUEUE = empty queue
  PUSH source TO QUEUE
  VISITED[source] = true
  parent[source] = -1

  WHILE QUEUE is not empty DO
    u = POP QUEUE
```

```
        FOR v FROM 0 TO V - 1 DO
            IF VISITED[v] == false AND residualGraph[u][v] > 0 THEN
                PUSH v TO QUEUE
                parent[v] = u
                VISITED[v] = true

    RETURN VISITED[sink] == true

ALGORITHM EdmondsKarp(graph, source, sink)
    residualGraph = COPY(graph)
    maxFlow = 0
    WHILE BFS(residualGraph, source, sink, parent) DO
        pathFlow = INFINITY
        s = sink
        WHILE s != source DO
            u = parent[s]
            pathFlow = MIN(pathFlow, residualGraph[u][s])
            s = parent[s]
        s = sink
        WHILE s != source DO
            u = parent[s]
            residualGraph[u][s] -= pathFlow
            residualGraph[s][u] += pathFlow
            s = parent[s]
        maxFlow += pathFlow

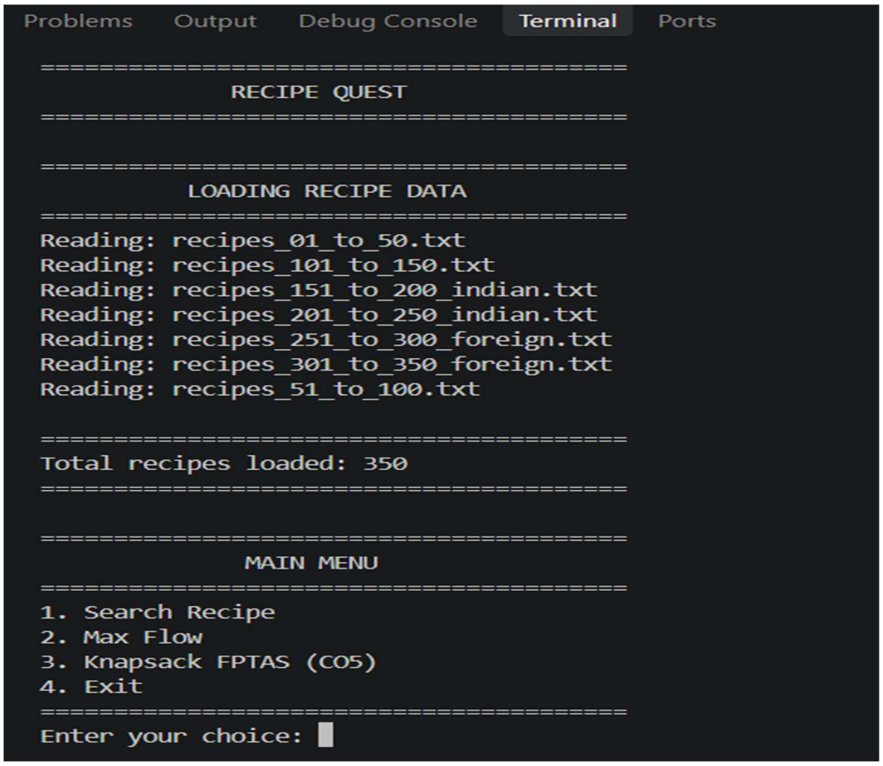
    RETURN maxflow
```

5. RESULTS

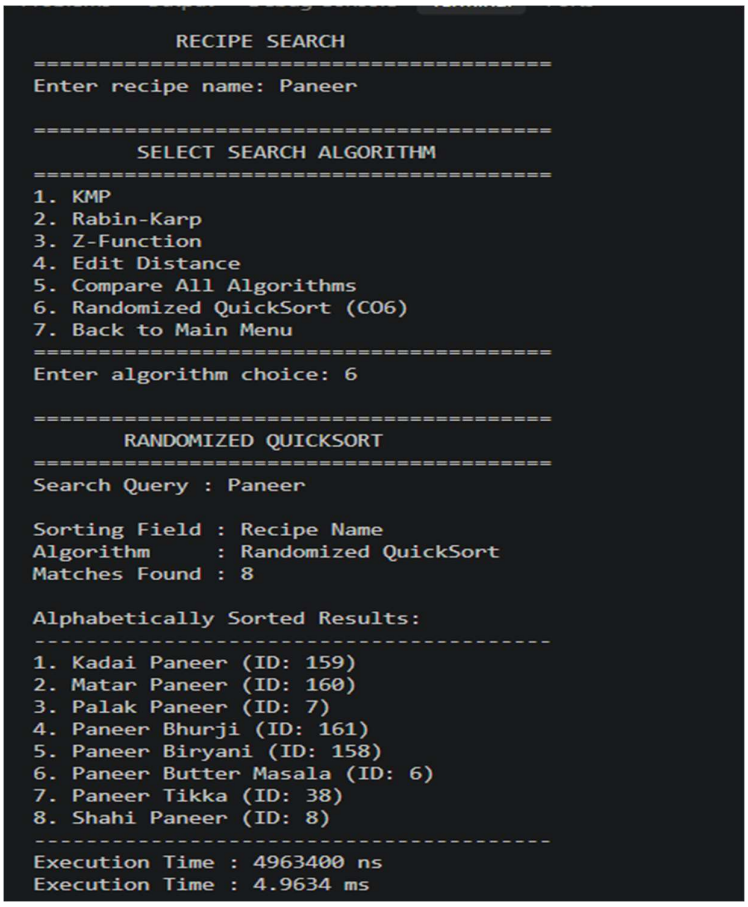
The implemented system was tested with various inputs and datasets:

Module/Feature	Input Test Case	Expected Result	Actual Result	Status
KMP Search	Pattern: "Paneer", Text: "Paneer Tikka Masala"	Pattern detected at index 0	Match found at index 0	PASS
Rabin-Karp Search	Pattern: "Chicken", Text: "Butter Chicken"	Substring matched using hashing	Match found at index 7	PASS
0/1 Knapsack	5 Recipes, Calorie Budget = 800 kcal	Maximum satisfaction subset	Optimal meal plan output	PASS
Edmonds-Karp	Source = 0, Sink = 5, Graph Capacities	Max Flow calculated correctly	Flow bottleneck computed	PASS

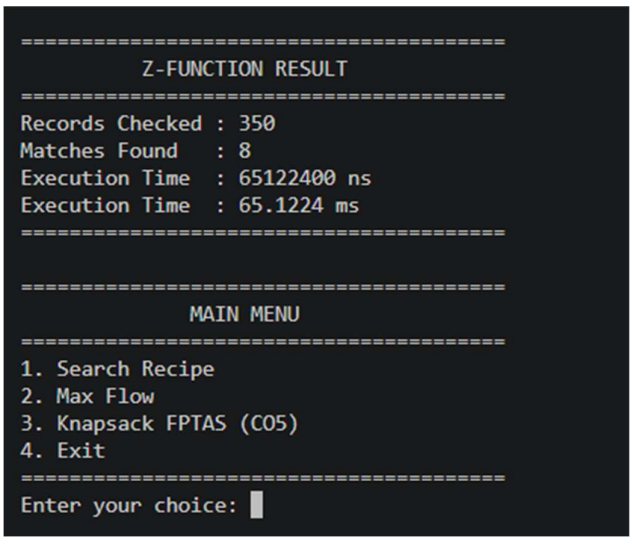
Main Menu & Recipe Ingestion



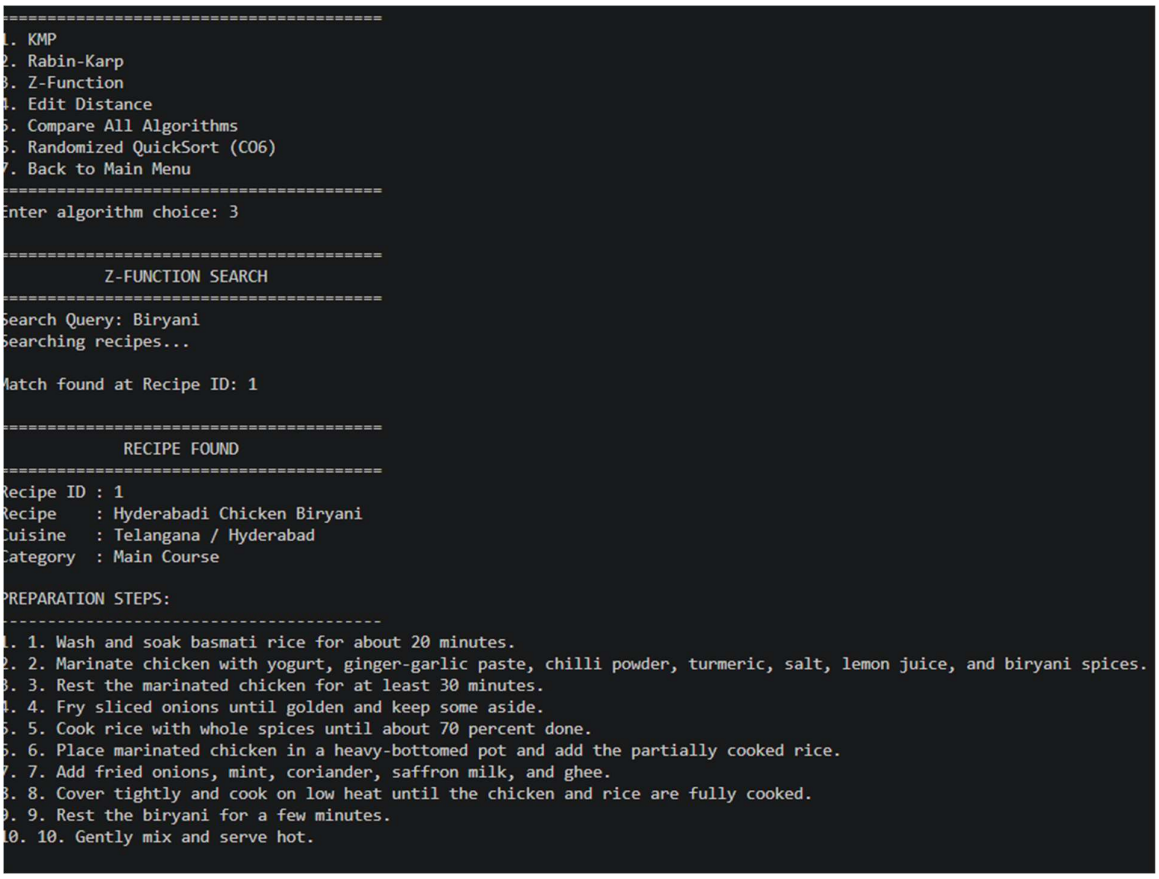
Randomized QuickSort Search Output



Z-Function Algorithm Execution Time



Detailed Recipe Display Result



6. CHALLENGES, LIMITATIONS & SOLUTIONS

6.1 Issues Faced During Development

- File Parsing Errors: Unstructured text formatting in recipe datasets caused index-out-of-bounds exceptions when extracting numerical calories/ratings.
- Graph Capacity Mappings: Correctly mapping non-integer ingredient units into edge capacities for network flow.

6.2 Solutions Applied

- **Robust File Readers**: Added string sanitization and default fallback values in RecipeReader.java.
- **Data Normalization**: Normalized caloric intake and rating values into clean integer arrays before running DP and Graph algorithms.

6.3 Limitations of the System

- **In-Memory Storage**: Data is processed in-memory without persistent database storage.
- **Console Interface**: The current user interface is command-line based.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

The Recipe Finder System demonstrates the practical application of core Data Structures and Algorithms - 3 concepts in real-world software development. By combining KMP/Rabin-Karp string matching with Dynamic Programming and Network Flow algorithms, the system delivers fast searches and efficient meal optimization.

7.2 Future Work

- **GUI Development**: Build a web dashboard using React/Vite.
- **Database Integration**: Connect to a PostgreSQL/MongoDB backend for dynamic data persistence.
- **Machine Learning**: Add recommendation systems based on user dietary preference history.

7.3 Skills Learned

- Advanced String Search Algorithms (KMP, Rabin-Karp).
- Dynamic Programming formulation for 0/1 Knapsack optimization.
- Graph theory and Edmonds-Karp Network Flow analysis.
- Modular Java OOP architecture and dataset parsing.

REFERENCES

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.
2. E. Horowitz, S. Sahni, and S. Rajasekaran, *Fundamentals of Computer Algorithms*, Silicon Press, 2008.
3. Oracle Java SE Documentation: https://docs.oracle.com/en/java/